Programowanie współbieżne w C++

Lista zadań nr 6

Na pracownię 9 kwietnia 2026

Zadanie 1 (4pkty) Implementacja zmiennych warunkowych.

Stwórz uproszczoną implementację zmiennej warunkowej, tzn. klasę `MyConditionVariable` z metodami:

```
void wait(std::unique_lock<std::mutex>& lock);
void notify_one() noexcept;
void notify_all() noexcept;
```

Użyj zmiennych atomowych (np. `std::atomic<uint32_t>`) z operacjami wait()/notify_one()/notify_all(). Przetestuj swoją implementację, np. używając jej w rozwiązaniu problemu producenta-konsumenta.

Zadanie 2 (2pkty) Analiza implementacji kolejki dwuwątkowej.

Posługując się definicjami relacji sequenced-before, synchronizes-with oraz happens-before udowodnij, że Twoja implementacji kolejki dwuwątkowej z poprzedniej listy jest wątkowo bezpieczna. Wszystkie operacje atomowe na tej kolejce przeprowadzamy w porządku domyślnym (czyli memory_order_seq_cst). Rozwiązanie zapisz w postaci raportu w pliku tekstowym.

Zadanie 3 (2pkty) Analiza implementacji puli wątków.

Powtórz poprzednie zadanie dla swojej implementacji puli wątków z poprzedniej listy.

Zadanie 4 (6pktów) Kolejka typu work-stealing queue bez blokad.

Zaimplementuj kolejkę bez blokad bazującą na tablicy elementów z góry określonego rozmiaru. Kolejka ma dwa końce, określone przez indeksy `top` i `bottom`. Właścicielem kolejki jest **pojedynczy** wątek, który może wykonać na niej operacje `push` lub `pop`, operujące na indeksie `bottom`. Oprócz właściciela, możemy mieć **dowolną liczbę wątków** złodziei, którzy podkradają

elementy z końca kolejki określonego przez indeks `top`. Zaimplementuj metody:

- `void push(T item)` – wywoływana tylko przez właściciela. Dodaje element na koniec kolejki (indeks `bottom`) i przesuwa indeks.
- `std::optional<T> pop()` – wywoływana tylko przez właściciela. Próbuje pobrać element z końca kolejki (zmniejsza `bottom`). Musi poprawnie obsłużyć sytuację konfliktu, gdy w kolejce został tylko jeden element i może on zostać skradziony.
- `std::optional<T> steal()` – wywoływana przez złodziei. Próbuje ukraść element z początku kolejki (indeks `top`). Złodzieje konkurują między sobą oraz z właścicielem, dlatego metoda ta wymaga użycia `compare_exchange`.

Zanim przystąpisz do implementacji dopracuj mechanizm dla zmian indeksów `top` i `bottom` w powyższych metodach. Uwzględnij przypadki brzegowe: kolejki pustej, prawie pustej (jednoelementowej), pełnej i prawie pełnej. Rzetelnie przetestuj swoją implementację!